

SESHADRIPURAM EDUCATIONAL TRUST
Seshadripuram College
Post Graduate Department of Computer Applications
#27, Nagappa Street, Seshadripuram, Bengaluru - 560020

PROJECT REPORT ON

AUTOMATED QUALITY ASSURANCE FRAMEWORK
FOR UI AND API TESTING

End-to-End Test Automation using Playwright and TypeScript

Submitted in partial fulfillment for the award of degree in
MASTER OF COMPUTER APPLICATIONS (MCA)
IV SEMESTER

Submitted by
Pradeep Kumar H N - P18GH24S126076

Under the Guidance of
Prof. VIJAYALKSHMI S KATTI

Academic Year: 2025-26

CERTIFICATE

This is to certify that the major project entitled **AUTOMATED QUALITY ASSURANCE FRAMEWORK FOR UI AND API TESTING**, carried out by Pradeep Kumar H N (P18GH24S126076), has been completed in the Post Graduate Department of Computer Applications, Seshadripuram College, during the academic year 2025-26, under the guidance of Prof. VIJAYALKSHMI S KATTI.

Project Guide _____

Prof. VIJAYALKSHMI S KATTI

Incharge Head _____

Prof. Rajeswari. V

Date: _____

DECLARATION

I, Pradeep Kumar H N - P18GH24S126076, hereby solemnly declare that the major project entitled **AUTOMATED QUALITY ASSURANCE FRAMEWORK FOR UI AND API TESTING** is a bonafide work carried out by me under the guidance of Prof. VIJAYALKSHMI S KATTI, Post Graduate Department of Computer Applications, Seshadripuram College, Bengaluru.

This project is my original work and has not been submitted for the award of any other degree or diploma from any other university. It is submitted in partial fulfillment of the requirements for the degree of Master of Computer Applications (MCA).

Place: Bengaluru

Date: _____

Pradeep Kumar H N
P18GH24S126076

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and faculty of Seshadripuram College for providing the academic environment and support required to complete this project. I thank the Principal, the Incharge Head of the Post Graduate Department of Computer Applications, and my project guide, Prof. VIJAYALKSHMI S KATTI, for their guidance, suggestions, and encouragement.

I also thank my classmates, friends, and family for their support throughout the planning, implementation, testing, and documentation of AutoQA Framework.

Pradeep Kumar H N
P18GH24S126076

ABSTRACT

This project presents AutoQA Framework, a maintainable end-to-end test automation framework for web user interfaces and REST APIs. The framework is implemented with TypeScript and Playwright and is organized around reusable browser actions, API actions, page objects, environment configuration, test fixtures, and multiple reporting adapters.

The framework targets the DemoQA web application for functional browser testing and uses both a local JSON Server employee service and the DemoQA Swagger Book Store API for API validation. Browser tests are configured for Chrome, Firefox, and WebKit, while API tests run through dedicated Playwright projects. The configuration supports QA, development, QA API, and development API environments without changing test source code.

The solution also demonstrates reporting and diagnostics through the Playwright HTML report, Allure, Ortoni HTML reporting, screenshots, video, traces, and framework logs. A GitHub Actions workflow installs Node.js 20 and Playwright dependencies, executes UI and API suites, generates reports, and uploads artifacts. The result is a reusable foundation that reduces duplicated automation code and provides a consistent execution model for local development and continuous integration.

TABLE OF CONTENTS

Chapter 1 - Introduction

1.1 Background and motivation

1.2 Characteristics of a good automation framework

1.3 Applications of automated quality assurance

Chapter 2 - Literature Review

2.1 Existing approaches and tools

2.2 Technology survey

2.3 Limitations of existing approaches

2.4 Proposed framework

Chapter 3 - Methodology

3.1 Problem statement

3.2 Objectives

3.3 Scope and modules

3.4 Out-of-scope components

Chapter 4 - Requirements Specification

4.1 Hardware and software requirements

4.2 Functional requirements

4.3 Non-functional requirements

4.4 Development and execution environment

Chapter 5 - Analysis and Design

5.1 System architecture

5.2 Test execution flow

5.3 Page object and action design

5.4 Reporting and CI design

Chapter 6 - Implementation and Results

6.1 Framework implementation

6.2 UI test implementation

6.3 API test implementation

6.4 Configuration, data, and diagnostics

6.5 Continuous integration

6.6 Results and generated artifacts

Chapter 7 - Testing

7.1 Test strategy

7.2 Coverage matrix

7.3 Defect diagnostics and evidence

Chapter 8 - Conclusion and Future Enhancements

8.1 Summary

8.2 Future enhancements

8.3 Limitations

REFERENCES

CHAPTER 1 - INTRODUCTION

1.1 Background and motivation

Modern web applications combine browser workflows, asynchronous network calls, third-party services, and responsive user interfaces. Manual regression testing of such applications is repetitive, slow, and vulnerable to inconsistent execution. A test automation framework addresses this problem by providing repeatable test execution, reusable abstractions, diagnostics, and machine-readable results.

AutoQA Framework was developed as a practical Playwright and TypeScript solution for validating both browser interactions and API contracts. The project uses DemoQA as the public UI and Book Store target and a local JSON Server database as a deterministic CRUD target. This combination demonstrates how one framework can validate user-visible behaviour and service-level behaviour while keeping environment-specific details in configuration.

1.2 Characteristics of a good automation framework

A good framework should be readable, modular, deterministic, configurable, diagnosable, and suitable for continuous integration. Tests should express business intent rather than low-level selector and request mechanics. Common actions should be centralized, page-specific locators should be isolated in page objects, and failures should preserve enough evidence for rapid investigation.

The framework addresses these characteristics through BaseTest, WebActions, APIActions, page repository classes, Playwright projects, environment maps, custom reporting, and failure artifacts. It also avoids a single hard-coded execution path by allowing the same test suite to run against different configured environments.

1.3 Applications of automated quality assurance

The framework can be used for smoke testing after deployment, functional regression testing, API contract checks, cross-browser compatibility testing, release verification, and continuous integration gates. Its reporting outputs are suitable for developers, testers, and project reviewers because they include test status, timing, attachments, traces, and execution context.

CHAPTER 2 - LITERATURE REVIEW

2.1 Existing approaches and tools

Traditional UI automation commonly uses Selenium-style driver abstractions, while API validation may use separate tools such as Postman collections or standalone HTTP clients. These tools are useful, but maintaining separate languages, runners, configuration files, and reporting pipelines can increase duplication. Playwright provides browser automation, request contexts, parallel projects, tracing, screenshots, and a unified test runner in one ecosystem.

2.2 Technology survey

TypeScript supplies static typing and editor support for test code. Playwright Test supplies fixtures, projects, retries, reporters, browser contexts, and API request support. JSON Server supplies a small local REST service for deterministic CRUD testing. Allure and Ortoni provide additional result views, while GitHub Actions supplies a repeatable cloud execution environment.

2.3 Limitations of existing approaches

Large end-to-end suites can become slow when every test starts from a full browser flow. Public websites can also change selectors or be affected by network availability. Test data can become polluted when API tests modify shared records. A framework must therefore separate environments, isolate test data where practical, retain diagnostics, and use stable abstractions without hiding real failures.

2.4 Proposed framework

The proposed solution combines reusable action classes, page objects, environment-driven configuration, independent UI and API projects, local mock data, multi-reporter output, and CI artifact publication. The design is intentionally transparent: test files remain visible and traceable to the application behaviour they verify.

CHAPTER 3 - METHODOLOGY

3.1 Problem statement

The project addresses the need for a single, maintainable automation framework that can validate UI workflows and REST APIs across browsers and environments while producing useful evidence when a check fails.

3.2 Objectives

The objectives are to build a TypeScript Playwright framework; implement reusable UI and API abstractions; cover representative DemoQA workflows; validate local CRUD and Swagger Book Store APIs; support Chrome, Firefox, and WebKit projects; provide environment configuration; generate HTML, Allure, and Ortoni reports; preserve screenshots, videos, and traces on failure; and execute the same checks in GitHub Actions.

3.3 Scope and functional modules

The scope includes browser navigation, form interaction, element validation, alerts and frames, widgets, login and book-store workflows, HAR-backed behaviour, employee CRUD operations, Swagger Book Store operations, configuration, reporting, and CI execution. The principal modules are the test runner configuration, shared action libraries, page repository, functional test suites, API test suites, mock database, reporters, and workflow automation.

3.4 Out-of-scope components

The project does not implement performance/load testing, mobile-native automation, penetration testing, production database integration, visual baseline governance, or a business application under test. Those capabilities can be integrated later without changing the core execution model.

CHAPTER 4 - REQUIREMENTS SPECIFICATION

4.1 Hardware and software requirements

A developer machine requires a modern multi-core processor, at least 8 GB RAM, sufficient storage for Node modules and browser binaries, Windows or Linux, Node.js 20 or later, npm, and the Playwright browser dependencies. CI uses an Ubuntu runner and installs the required browser dependencies before execution.

4.2 Functional requirements

FR1: The framework shall execute functional UI tests against the configured base URL. FR2: It shall support Chrome, Firefox, and WebKit browser projects. FR3: It shall execute local employee CRUD API tests. FR4: It shall execute Swagger Book Store API tests. FR5: It shall select environment URLs through the ENV setting. FR6: It shall generate HTML and additional test reports. FR7: It shall retain failure diagnostics. FR8: It shall run in GitHub Actions and upload generated artifacts.

4.3 Non-functional requirements

The framework shall be readable, reusable, type-safe, maintainable, diagnosable, repeatable, and extensible. It shall fail explicitly when an invalid environment is selected. It shall keep test-specific selectors and request details close to their domain abstractions and shall avoid coupling API tests to a running external application when a local mock service is sufficient.

4.4 Development and execution environment

The package manifest identifies @playwright/test, TypeScript, cross-env, json-server, start-server-and-test, allure-playwright, allure-commandline, orton-report, Winston, and CryptoJS. The supported environment keys are qa, dev, qaApi, and devApi. The QA UI endpoint is <https://demoqa.com> and the local QA API endpoint is <http://127.0.0.1:3000>.

CHAPTER 5 - ANALYSIS AND DESIGN

5.1 System architecture

The framework is organized into six layers. The test layer contains functional and API specifications. The page repository contains page-specific locators and workflows. Shared action libraries provide browser and request operations. Configuration maps environment names to URLs and execution options. The Playwright runner composes projects and reporters. The CI layer provisions dependencies, runs the suites, and publishes artifacts.

5.2 Test execution flow

Execution begins with ENV selection and configuration validation. Global setup runs before tests. Playwright creates the selected browser or API project, loads the appropriate base URL, and executes matching specifications. On failure, screenshots, videos, traces, and logs are retained according to configuration. Reporters serialize results into HTML, Allure, and Ortoni outputs.

5.3 Page object and action design

Page classes such as `ElementsPage`, `InteractionsPage`, `LoginPage`, `WidgetsPage`, and `AlertsFrameWindowsPage` isolate locators and domain actions. `WebActions` centralizes common navigation and interaction operations. `APIActions` centralizes request methods and response handling. `BaseTest` provides common test setup and reduces repeated fixture code.

5.4 Reporting and CI design

The reporter configuration emits the standard Playwright HTML report, Allure results, an Ortoni report, and custom reporter output. The workflow runs UI tests, starts the mock API through the test script, runs API tests, generates Allure output even when earlier steps fail, and uploads reports and test-results as artifacts.

CHAPTER 6 - IMPLEMENTATION AND RESULTS

6.1 Framework implementation

The implementation uses a typed Playwright configuration with explicit environment validation. Browser projects are generated for Chrome, Firefox, and WebKit. Separate API and SwaggerAPI projects match `MockCrud.test.ts` and `SwaggerBookStore.test.ts`. Browser contexts use a 1920 by 1080 viewport, headless execution, failure screenshots, retained failure video, and retained failure traces.

6.2 UI test implementation

The functional suite covers AlertsFrameWindows, Elements, HAR, Interactions, Login, and Widgets. These tests exercise navigation, forms, checkboxes, radio buttons, web tables, browser alerts, frames, windows, widgets, login flows, and book-store interactions against DemoQA. Page repository classes keep selectors and workflows separate from assertions.

6.3 API test implementation

`MockCrud.test.ts` validates GET collection, GET by identifier, POST, PUT, PATCH, and DELETE operations against the JSON Server employee resource. `SwaggerBookStore.test.ts` validates the DemoQA Book Store API and temporary-user book management flow. The API projects run independently of browser projects and can use a one-worker mode when deterministic sequencing is needed.

6.4 Configuration, data, and diagnostics

`testConfig.ts` stores environment endpoints and execution settings. `mock-db.json` provides local employee data. `global-setup.ts` performs test-run initialization. The repository includes HTML reports, Allure results, screenshots, videos, traces, and logs, allowing a failed assertion to be investigated without immediately reproducing it.

6.5 Continuous integration

The GitHub Actions workflow triggers on pushes and pull requests targeting main. It checks out the repository, installs Node.js 20 and npm dependencies, installs Chrome with dependencies, runs UI and API suites, generates Allure results, and uploads Ortoni, Allure, and test-result artifacts. This makes quality checks repeatable and reviewable.

6.6 Results and generated artifacts

The workspace contains generated html-report and allure-report directories and test-results evidence from Playwright execution. The report artefacts include structured result data, screenshots, traces, and web recordings where available. These artifacts demonstrate that the framework is not limited to source code: it also provides an operational feedback loop for test execution and diagnosis.

CHAPTER 7 - TESTING

7.1 Test strategy

Testing is organized by system boundary and execution purpose. UI tests validate behaviour through a real browser, API tests validate service contracts directly, and CI validates that the same commands work in a clean runner. Smoke-tagged tests can be selected with `--grep @Smoke` and can be run in parallel or serial mode.

7.2 Coverage matrix

UI coverage includes elements, interactions, alerts and frames, widgets, login, and HAR-backed flows. API coverage includes local employee collection and identifier operations plus Swagger Book Store workflows. Cross-browser coverage is configured for Chrome, Firefox, and WebKit. Environment coverage is represented by QA and API-specific configuration keys, with development keys available for extension.

7.3 Defect diagnostics and evidence

The framework uses explicit environment validation, centralized timeout configuration, failure-only screenshots, retained failure video and traces, console and request reporting through reporters, and CI artifact retention. These mechanisms reduce the time between a failed check and a reproducible diagnosis. The generated reports in the workspace provide evidence for reviewing execution history and failure attachments.

CHAPTER 8 - CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Summary

AutoQA Framework provides a coherent, reusable, and CI-ready approach to end-to-end UI and API automation. Playwright and TypeScript form the execution foundation, while page objects, shared action classes, typed configuration, local mock data, and multiple reporters provide maintainability and observability. The framework demonstrates the practical value of keeping UI and API checks under one test runner.

8.2 Future enhancements

Future work may add visual regression baselines, accessibility checks, schema validation, contract testing, test-data factories, database assertions, parallel sharding, environment secrets through CI storage, richer dashboard metrics, and a formal test-case traceability matrix. Additional device profiles and mobile browser projects can extend coverage.

8.3 Limitations

Public DemoQA tests depend on external availability and may change independently of the framework. Browser binaries and network access are required for full execution. The local JSON Server is intentionally lightweight and does not model production authentication, concurrency, or database constraints. The current project is a test automation framework and not a production application.

The project meets its central objective: it supplies a structured automation foundation that can be executed locally and in CI, covers representative UI and API scenarios, and produces evidence that supports engineering decisions.

REFERENCES

1. Microsoft Playwright Documentation, <https://playwright.dev/docs/intro>
2. Playwright Test Configuration Documentation, <https://playwright.dev/docs/test-configuration>
3. TypeScript Documentation, <https://www.typescriptlang.org/docs/>
4. GitHub Actions Documentation, <https://docs.github.com/en/actions>
5. JSON Server Documentation, <https://github.com/typicode/json-server>
6. Allure Report Documentation, <https://allurereport.org/docs/>
7. DemoQA, <https://demoqa.com/>
8. Project source code and generated execution artifacts in the AutoQAFramework repository.